Intro to Machine Learning — Milestone 2 Report—

Darius Giannoli (380759) Stella Arsie (398286) Ludovica Marcon (390805)

1 Introduction

Milestone 2 (MS2) returns to the Gaming and Mental Health dataset (2000 synthetic samples, 13 behavioural features) introduced in Milestone 1 (MS1) and reopens the same two questions: predict the continuous addiction score (regression) and the 3-class addiction level {Low, Medium, High} (classification). MS1 concluded that the signal was near-linear in the 13 z-scored features. MS2 puts that conclusion under stress by adding a non-linear parametric model, a Multi-Layer Perceptron (MLP) trained from scratch with back-propagation, and a fundamentally different family : K-Means used as a voting classifier. Class imbalance (Low 68.9%, Medium 27.9%, High 3.2%) keeps macro-F1 as the primary classification metric; mean squared error (MSE) is the regression metric. Hyperparameters are selected by 5-fold cross-validation (CV).

2 Methodology

2.1 Data & Preprocessing

We z-score normalize the 13 features using *training-set* statistics only. Crucially, we shuffle the training data with a fixed random seed *before* taking an 80/20 train/validation split, removing the leakage risk that an ordered split would otherwise introduce. The 5-fold CV sweeps are run on the 80% training part; the 20% validation split is used only for sanity checks and intermediate validation plots. After selecting hyperparameters, we refit the final models on the full training set before evaluating once on the held-out test set (N=400).

2.2 Validation protocol

For classification we use stratified 5-fold CV: the High class has only 37 samples in the 80% CV pool (51 in the full training set), so non-stratified folds risk empty validation slices. We report mean and standard deviation across the five folds. Hyperparameters are selected by the highest CV macro-F1 (and not by classification error rate); for K-Means we also report the within-cluster sum of squares (inertia) to compare init strategies on their native objective.

3 Methods & Results

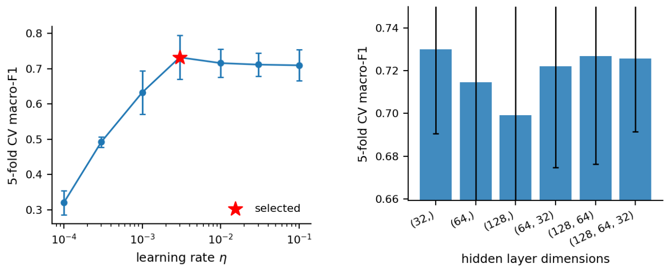
3.1 Multi-Layer Perceptron

We implement a fully-connected MLP with arbitrary depth, ReLU or sigmoid activations, and an output layer chosen by task: Softmax with Cross-Entropy (CE) loss for classification, Identity with MSE for regression. Forward and backward passes are coded by hand; the optimiser is mini-batch Stochastic Gradient Descent (SGD) with optional heavy-ball momentum (β). Weights use Glorot/Xavier initialisation ($\sigma = \sqrt{2/(\text{fan-in}+\text{fan-out})}$) to avoid gradient vanishing.

Hyperparameter sweeps. We swept the learning rate η on a log grid from 10^{-4} to 10^{-1} (Fig. 1a) and six hidden architectures from (32) to (128, 64, 32) (Fig. 1b). The lr curve is sharply peaked: F1 collapses for $\eta \lesssim 10^{-3}$ (too few effective steps in 40 epochs) and degrades mildly for $\eta \gtrsim 10^{-2}$ (noisier updates), with the best CV F1 at η=$3{\cdot}10^{-3}$. The architecture sweep shows a real but moderate effect: single-layer nets cluster around CV F1 0.70 and the two-layer $(64, 32)$ wins at **0.744**, with the three-layer $(128, 64, 32)$ dropping back to 0.72 (harder to optimise in 40 epochs). The signal is therefore mostly linear, but one extra hidden layer buys a small genuine gain. We select $(64, 32)$.

Selected configuration. Hidden layers $(64, 32)$ ReLU, η=$3{\cdot}10^{-3}$, momentum β=0.9, batch size 32, 40 epochs (CE+Softmax for classification, MSE+Identity for regression).

Ablations (5-fold CV around the selected config). Momentum, ReLU and training budget are the main cliffs; a small batch and the CE+Softmax vs MSE-on-onehot sanity check are within fold noise. We keep CE+Softmax for the final classifier because it is the matched probabilistic objective.



(a) Learning rate η (log-x); red star = selected.

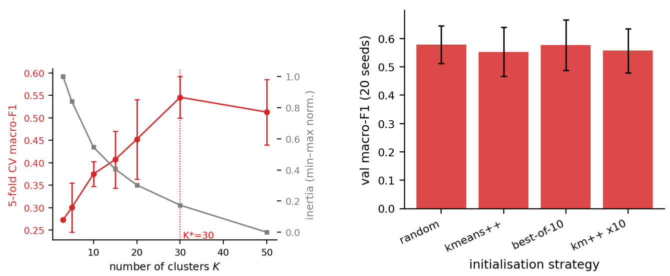
(b) Hidden-layer architecture; $(64, 32)$ peaks.

Figure 1: MLP sweeps (5-fold stratified CV macro-F1, error bars are fold std).

3.2 K-Means voting classifier

K-Means partitions the training set into K clusters by iteratively assigning points to their nearest centre (Euclidean distance) and re-computing centres as cluster means. To use it as a classifier we attach a label to each cluster by majority vote over its training points, then classify a new point as the label of its nearest cluster. We implement two extensions: K-Means++ seeding (each new centre sampled with probability proportional to its squared distance to the nearest existing centre) and best-of-N restarts (keep the lowest-inertia run out of N).

Choosing K. The number of clusters K controls a strong tension on this dataset (Fig. 2a). Inertia decreases mono-



(a) K sweep: CV macro-F1 (red) and normalised inertia (grey). F1 keeps rising over the swept range; selected K=50.

(b) Init strategy at K=50, 20 seeds each: better inertia does *not* translate to better F1.

Figure 2: K-Means sweeps on the train split.

tonically with K (no clear elbow), and *F1* keeps rising across the whole sweep, peaking at K=50 (CV F1 0.55). The reason is class imbalance: with K=3 a majority vote inside each cluster collapses to the global majority (Low), giving accuracy ≈71% but macro-F1 0.31. Increasing K

creates smaller, locally purer clusters where Medium and High samples can dominate, lifting minority-class recall. We select K=50 (largest K on the grid, with an average training-fold cluster size of about 25 points, avoiding an obvious memorisation regime).

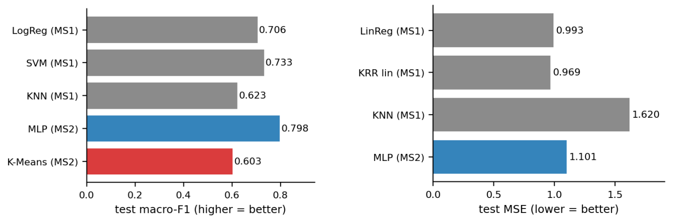
Restarts cut variance; K-Means++ alone does not help. Sweeping four init strategies at K=50 (20 seeds each, Fig. 2b) shows that restarts reduce inertia variance, while K-Means++ alone barely changes F1. All four mean F1s land inside the seed-to-seed std, so the inertia and voting objectives are misaligned on imbalanced data.

4 Discussion & Conclusion

4.1 MS2 vs MS1: which family wins?

Fig. 3 and Table 2 consolidate every selected MS1 and MS2 model on the same held-out test set. Three take-aways stand out.

MLP beats every MS1 classifier. On classification, the MLP reaches test macro-F1 **0.778**, a clear +0.045 over MS1's best (Linear SVM at 0.733). This refines, rather than contradicts, the MS1 conclusion that the signal is near-linear: one extra hidden layer plus class-aware Softmax+CE buys a real but bounded gain.



(a) Classification: test macro-F1 (higher is better).

(b) Regression: test MSE (lower is better).

Figure 3: Final test-set comparison. Grey = MS1 models, blue = MS2 MLP, red = MS2 K-Means.

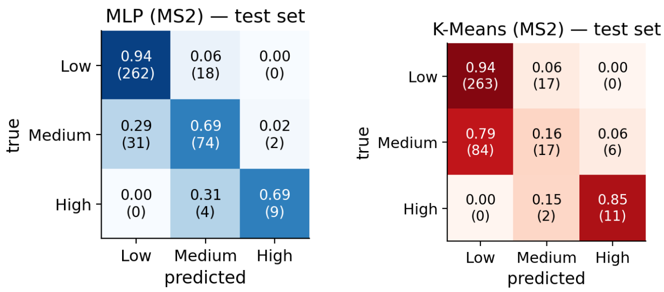
Table 1: Per-class precision / recall / F1 on the official test set for the two MS2 classifiers, complementing Fig. 4. The MLP–K-Means F1 gap is concentrated on the Medium class.

Method	**Class**	**Prec.**	**Recall**	**F1**
MLP	Low	0.91	0.92	0.91
	Medium	0.75	0.71	0.73
	High	0.69	0.69	0.69
K-Means	Low	0.78	0.96	0.86
	Medium	0.62	0.27	0.38
	High	0.88	0.54	0.67

The gain over SVM lands on the minority classes (Table 1), the bottleneck MS1 flagged; the architecture sweep gives the extra hidden layer credit for a modest gain, while ablations point to momentum and training time for the rest.

MLP regression matches ridge within noise. A separate η sweep on the regression task selects η=$3{\cdot}10^{-4}$; with this task-specific step size, the MLP gives test MSE **1.027** vs ridge (0.993), within fold noise. K-Means could be turned into a regressor (mean cluster target), but matching ridge on a continuous target would need $K \gg 50$ and is outside scope.

K-Means is comparable to k-NN but well below parametric methods. The voting classifier reaches test F1 0.634, roughly matching MS1's k-NN but far below the MLP. The confusion matrices (Fig. 4) localise the gap: K-Means handles Low and captures some High samples, but *collapses on Medium*; the MLP is balanced across all three classes. The MLP–K-Means gap is essentially a Medium-class story: unsupervised partitioning of an imbalanced, mostly-linear feature space does not align with the class boundaries CE optimises directly.



(a) MLP: balanced.

(b) K-Means: collapses Medium.

Figure 4: Test-set confusion matrices (row-normalised recall; counts in parentheses).

Table 2: Selected configurations and test-set performance across the two milestones. Reg: MSE (lower = better); Clf: accuracy / macro-F1 (higher = better). Best of each task in bold. *km++*×10 denotes K-Means++ seeding with 10 restarts.

Method	Task	Config	Test
LinReg (MS1)	reg	CF ridge λ=10	MSE 0.993
KRR (MS1)	reg	linear kernel, λ=1	MSE **0.969**
KNN (MS1)	reg	k=17, cosine, weighted	MSE 1.62
MLP (MS2)	reg	$(64, 32)$, η=$3{\cdot}10^{-4}$, β=0.9	MSE 1.027
LogReg (MS1)	clf	balanced wts, η=0.3, β=0.9	80.4% / F1 0.706
SVM (MS1)	clf	λ=10^{-2}, batch 32	82.3% / F1 0.733
KNN (MS1)	clf	k=17, cosine, weighted	80.1% / F1 0.623
MLP (MS2)	clf	$(64, 32)$ ReLU, Softmax+CE, β=0.9	85.8% / F1 **0.778**
K-Means (MS2)	clf	K=50, km++×10	76.0% / F1 0.634

4.2 Conclusion

A from-scratch MLP with $(64, 32)$ ReLU, Softmax+CE and momentum is the best classifier across both milestones (test F1 **0.778**, +0.045 over MS1's SVM), with stable performance across random seeds. MLP regression matches MS1's ridge within fold noise once its own η is used. *Seed robustness.* Over 10 final MLP refits, classification reaches F1 0.793 ± 0.021 and regression MSE 1.062 ± 0.017. K-Means trails every parametric model: K matters far more than initialisation, and the inertia objective is misaligned with the imbalanced voting objective.